

Striver SDE Sheet — Arrays Part II

1. Rotate Matrix by 90° Clockwise

Problem

Given an $n \times n$ matrix, rotate it by 90° clockwise in place.

Example:

1 2 3		7 4 1
4 5 6	→	8 5 2
7 8 9		9 6 3

Approach 1: Brute Force — Use another matrix

Idea

For every element at position:

(i, j)

its position after clockwise rotation becomes:

$(j, n - 1 - i)$

Therefore:

```
answer[j][n - 1 - i] = matrix[i][j]
```

Algorithm

```
create answer[n][n]

for i from 0 to n - 1:
```

```
for j from 0 to n - 1:  
    answer[j][n - 1 - i] = matrix[i][j]
```

Copy `answer` back into the original matrix if required.

Complexity

- Time: $O(n^2)$
- Space: $O(n^2)$

Approach 2: Optimal — Transpose and reverse rows

A clockwise rotation can be divided into two operations:

1. Transpose the matrix
2. Reverse every row

Step 1: Transpose

Convert rows into columns:

```
matrix[i][j] ↔ matrix[j][i]
```

Only traverse above the main diagonal to avoid swapping twice.

```
for i from 0 to n - 1:  
    for j from i + 1 to n - 1:  
        swap(matrix[i][j], matrix[j][i])
```

Example:

```
1 2 3      1 4 7  
4 5 6 →    2 5 8  
7 8 9      3 6 9
```

Step 2: Reverse every row

1 4 7	→	7 4 1
2 5 8	→	8 5 2
3 6 9	→	9 6 3

Pseudocode

```
for i from 0 to n - 1:  
    for j from i + 1 to n - 1:  
        swap(matrix[i][j], matrix[j][i])  
  
for each row:  
    reverse(row)
```

Complexity

- Time: $O(n^2)$
- Extra space: $O(1)$

Revision key

Clockwise rotation = transpose + reverse every row.

2. Merge Overlapping Intervals

Problem

Given intervals:

```
[[1,3], [2,6], [8,10], [15,18]]
```

Merge overlapping intervals.

Output:

```
[[1,6], [8,10], [15,18]]
```

Approach 1: Brute Force

Idea

1. Sort intervals by starting time.
2. For every interval, check following intervals.
3. Continue merging while they overlap.
4. Add the merged interval to the answer.

Two intervals overlap when:

```
nextStart <= currentEnd
```

Algorithm

```
sort intervals

for i from 0 to n - 1:
    start = intervals[i][0]
    end = intervals[i][1]

    if interval already covered:
        continue

    for j from i + 1 to n - 1:
        if intervals[j][0] <= end:
            end = max(end, intervals[j][1])
        else:
            break

    answer.push([start, end])
```

Complexity

- Time: $O(n \log n + n^2)$
- Space: $O(n)$ for output

Approach 2: Optimal — Single traversal after sorting

Core idea

After sorting by starting time, overlapping intervals appear next to one another.

Maintain the last interval inserted into the answer.

For each interval:

- If it does not overlap, add it.
- If it overlaps, extend the previous interval.

Algorithm

```
sort intervals by starting point

answer = empty list

for interval in intervals:

    if answer is empty
      or interval.start > answer.last.end:

        answer.push(interval)

    else:
      answer.last.end =
        max(answer.last.end, interval.end)
```

Example

```
Intervals:
[1,3], [2,6], [8,10], [9,12]
```

Process:

```
[1,3] → add

[2,6] overlaps because 2 <= 3
      → merge into [1,6]

[8,10] does not overlap because 8 > 6
      → add

[9,12] overlaps because 9 <= 10
      → merge into [8,12]
```

Answer:

```
[[1,6], [8,12]]
```

Complexity

- Sorting: $O(n \log n)$
- Traversal: $O(n)$
- Total time: $O(n \log n)$
- Space: $O(n)$ for answer

Edge case

Intervals touching at their boundaries are considered overlapping:

```
[1,4] and [4,5]
```

Because:

```
4 <= 4
```

They merge into:

```
[1,5]
```

Revision key

Sort intervals, then compare the current start with the previous merged end.

3. Merge Two Sorted Arrays Without Extra Space

Problem

Given two sorted arrays:

```
arr1 = [1,4,8,10]  
arr2 = [2,3,9]
```

Merge them while keeping both arrays sorted, without using another array.

Final result:

```
arr1 = [1,2,3,4]
arr2 = [8,9,10]
```

Approach 1: Brute Force — Extra array

Idea

Use the standard merge procedure from merge sort.

```
i = 0
j = 0
```

Compare elements from both arrays and place the smaller one into a third array.

Copy the first `n` elements back into `arr1` and the remaining `m` elements into `arr2`.

Complexity

- Time: `O(n + m)`
- Space: `O(n + m)`

Approach 2: Better — Swap and sort

Core idea

The largest elements of `arr1` may belong in `arr2`, while the smallest elements of `arr2` may belong in `arr1`.

Use:

```
left = n - 1
right = 0
```

Compare:

```
arr1[left] and arr2[right]
```

If the left value is greater, swap them.

Algorithm

```
left = n - 1
right = 0

while left >= 0 and right < m:

    if arr1[left] > arr2[right]:
        swap(arr1[left], arr2[right])
        left--
        right++
    else:
        break

sort(arr1)
sort(arr2)
```

Why can we stop when `arr1[left] <= arr2[right]`?

Both arrays are sorted.

Therefore, all remaining values on the left are smaller and all remaining values on the right are larger.

Complexity

- Swapping: `O(min(n,m))`
- Sorting: `O(n log n + m log m)`
- Space: `O(1)` depending on sorting implementation

Approach 3: Optimal — Gap Method

The gap method is inspired by Shell Sort.

Treat both arrays as one virtual array of size:

```
length = n + m
```

Compare elements that are `gap` positions apart.

Initial gap

```
gap = ceil((n + m) / 2)
```

Integer formula:

```
gap = (length / 2) + (length % 2)
```

After each pass:

```
gap = ceil(gap / 2)
```

Stop after processing `gap = 1`.

Three comparison cases

Let:

```
left = 0  
right = left + gap
```

Case 1: Both pointers are in `arr1`

```
left < n and right < n
```

Compare:

```
arr1[left] and arr1[right]
```

Case 2: Left is in `arr1`, **right is in** `arr2`

```
left < n and right >= n
```

Compare:

```
arr1[left] and arr2[right - n]
```

Case 3: Both pointers are in arr2

```
left >= n
```

Compare:

```
arr2[left - n] and arr2[right - n]
```

Swap whenever the left element is greater.

Pseudocode

```
length = n + m
gap = ceil(length / 2)

while gap > 0:

    left = 0
    right = left + gap

    while right < length:

        if left < n and right < n:
            compare arr1[left], arr1[right]

        else if left < n and right >= n:
            compare arr1[left], arr2[right - n]

        else:
            compare arr2[left - n], arr2[right - n]

        swap if left value > right value

        left++
        right++

    if gap == 1:
        break
```

```
gap = ceil(gap / 2)
```

Complexity

- Time: approximately $O((n + m) \log(n + m))$
- Extra space: $O(1)$

Revision key

Treat both arrays as one array and compare elements at decreasing gaps.

4. Find the Duplicate Number

Problem

An array contains $n + 1$ integers where every integer lies between 1 and n .

Exactly one number is repeated.

Example:

```
[1,3,4,2,2]
```

Output:

```
2
```

The array should not be modified, and ideally only constant extra space should be used.

Approach 1: Brute Force

For each element, check whether the same value appears later.

```
for i from 0 to n - 1:
    for j from i + 1 to n:
        if arr[i] == arr[j]:
            return arr[i]
```

Complexity

- Time: $O(n^2)$
 - Space: $O(1)$
-

Approach 2: Sorting

Sort the array and compare adjacent elements.

```
sort(arr)

for i from 0 to n - 1:
    if arr[i] == arr[i + 1]:
        return arr[i]
```

Complexity

- Time: $O(n \log n)$
- Space: depends on sorting

Limitation

Sorting modifies the input array.

Approach 3: Frequency array

Create a frequency array of size $n + 1$.

```
frequency[value]++
```

Return the number whose frequency becomes greater than one.

Complexity

- Time: $O(n)$
 - Space: $O(n)$
-

Approach 4: HashSet

Insert elements into a set.

If an element already exists, it is the duplicate.

Complexity

- Average time: $O(n)$
- Space: $O(1)$

Approach 5: Optimal — Floyd's Cycle Detection

Core observation

Treat each array value as a pointer to another index:

```
index → arr[index]
```

Because the array has $n + 1$ values but only values from 1 to n , at least two indices point to the same location.

This creates a cycle.

The entrance of the cycle represents the duplicate number.

Phase 1: Detect intersection inside the cycle

Use slow and fast pointers.

```
slow = arr[0]
fast = arr[0]

do:
    slow = arr[slow]
    fast = arr[arr[fast]]
while slow != fast
```

- Slow moves one step.
- Fast moves two steps.

They eventually meet inside the cycle.

Phase 2: Find the cycle entrance

Reset slow to the beginning:

```
slow = arr[0]
```

Now move both pointers one step at a time:

```
while slow != fast:  
    slow = arr[slow]  
    fast = arr[fast]
```

The position where they meet is the duplicate.

```
return slow
```

Example

```
arr = [1,3,4,2,2]
```

Pointer mapping:

```
0 → 1  
1 → 3  
3 → 2  
2 → 4  
4 → 2
```

Cycle:

```
2 → 4 → 2
```

Cycle entrance:

```
2
```

Therefore, duplicate:

2

Complexity

- Time: $O(n)$
- Extra space: $O(1)$
- Input is not modified

Revision key

Interpret array values as next pointers. The duplicate is the cycle entrance.

5. Find Repeating and Missing Number

Problem

An array contains numbers from 1 to n .

One number appears twice and one number is missing.

Example:

```
arr = [3,1,2,5,3]
```

Output:

```
Repeating = 3  
Missing = 4
```

Approach 1: Brute Force

For every number from 1 to n , count how many times it occurs.

```
for number from 1 to n:  
    count = 0  
  
    for value in array:  
        if value == number:  
            count++
```

```
if count == 2:
    repeating = number

if count == 0:
    missing = number
```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Frequency array

Create:

```
frequency[n + 1]
```

Count every value.

Then:

```
frequency[i] == 2 → repeating
frequency[i] == 0 → missing
```

Complexity

- Time: $O(n)$
- Space: $O(n)$

Approach 3: Optimal — Mathematical equations

Let:

```
x = repeating number
y = missing number
```


Equation 1: Difference of sums

Expected sum:

$$SN = n(n + 1) / 2$$

Actual array sum:

$$S$$

Because x appears twice and y is absent:

$$S - SN = x - y$$

Let:

$$\text{value1} = S - SN$$

Therefore:

$$x - y = \text{value1}$$

Equation 2: Difference of square sums

Expected square sum:

$$S2N = n(n + 1)(2n + 1) / 6$$

Actual square sum:

$$S2$$

Then:

$$S2 - S2N = x^2 - y^2$$

Using:

$$x^2 - y^2 = (x-y)(x+y)$$

Therefore:

$$x + y = (S2 - S2N) / (x - y)$$

Let:

$$\text{value2} = (S2 - S2N) / \text{value1}$$

Now:

$$\begin{aligned} x - y &= \text{value1} \\ x + y &= \text{value2} \end{aligned}$$

Add both equations:

$$2x = \text{value1} + \text{value2}$$

Therefore:

$$x = (\text{value1} + \text{value2}) / 2$$

And:

$$y = x - \text{value1}$$

Pseudocode

```
S = sum of array
S2 = sum of squares of array

SN = n(n + 1) / 2
S2N = n(n + 1)(2n + 1) / 6
```

```
value1 = S - SN
value2 = (S2 - S2N) / value1

repeating = (value1 + value2) / 2
missing = repeating - value1
```

Important

Use `long long` or a 64-bit integer because square sums can overflow an `int`.

Complexity

- Time: `O(n)`
- Extra space: `O(1)`

Approach 4: Optimal — XOR

This approach avoids arithmetic overflow.

Let:

```
repeating = x
missing = y
```

Step 1: XOR array elements and numbers from `1` to `n`

```
xr = 0

for i from 0 to n - 1:
    xr ^= arr[i]
    xr ^= (i + 1)
```

All normal numbers cancel.

Therefore:

```
xr = x XOR y
```

Step 2: Find one differing bit

Find the rightmost set bit:

```
bit = xr & (-xr)
```

This bit is different between `x` and `y`.

Step 3: Divide numbers into two groups

Place array values and numbers `1...n` into groups based on whether the selected bit is set.

```
zeroGroup = 0  
oneGroup = 0
```

```
for each array value:  
    if value & bit:  
        oneGroup ^= value  
    else:  
        zeroGroup ^= value  
  
for number from 1 to n:  
    if number & bit:  
        oneGroup ^= number  
    else:  
        zeroGroup ^= number
```

The two groups now contain:

```
x and y
```

But we do not yet know which is repeating.

Step 4: Identify repeating number

Count the occurrence of one candidate in the original array.

```
if zeroGroup occurs twice:  
    repeating = zeroGroup
```

```
    missing = oneGroup
else:
    repeating = oneGroup
    missing = zeroGroup
```

Complexity

- Time: $O(n)$
- Extra space: $O(1)$

Revision key

Sum equations are easier; XOR avoids overflow and separates the two numbers using a differing bit.

6. Count Inversions in an Array

Problem

An inversion is a pair of indices (i, j) such that:

$i < j$

but:

$arr[i] > arr[j]$

Example:

$arr = [5, 3, 2, 4, 1]$

Inversions:

$(5, 3)$
 $(5, 2)$
 $(5, 4)$
 $(5, 1)$
 $(3, 2)$
 $(3, 1)$

(2,1)
(4,1)

Total:

8

Approach 1: Brute Force

Check every pair.

```
count = 0

for i from 0 to n - 1:
    for j from i + 1 to n - 1:
        if arr[i] > arr[j]:
            count++
```

Complexity

- Time: $O(n^2)$
- Space: $O(1)$

Approach 2: Optimal — Modified Merge Sort

Core idea

Merge sort divides the array into two sorted halves.

Suppose:

```
left half = [2,3,5]
right half = [1,4]
```

While merging:

```
left[i] > right[j]
```

Because the left half is sorted, every element from `left[i]` to the end of the left half is also greater than `right[j]`.

Therefore, the number of inversions added is:

```
mid - i + 1
```

Merge logic

```
i = low
j = mid + 1
count = 0

while i <= mid and j <= high:

    if arr[i] <= arr[j]:
        temp.push(arr[i])
        i++

    else:
        temp.push(arr[j])
        count += mid - i + 1
        j++
```

Copy remaining values and place the merged result back into the original array.

Why `mid - i + 1`?

Suppose:

```
Left  = [3,5,7]
Right = [2,6]
```

When comparing:

```
3 > 2
```

Since the left half is sorted:

3, 5 and 7

are all greater than 2.

Therefore, three inversions are found at once:

$\text{mid} - i + 1 = 3$

Complete structure

```
mergeSort(arr, low, high):  
  
    if low >= high:  
        return 0  
  
    mid = (low + high) / 2  
  
    count = 0  
  
    count += mergeSort(arr, low, mid)  
    count += mergeSort(arr, mid + 1, high)  
    count += merge(arr, low, mid, high)  
  
    return count
```

Merge function

```
merge(arr, low, mid, high):  
  
    temp = empty list  
    left = low  
    right = mid + 1  
    count = 0  
  
    while left <= mid and right <= high:  
  
        if arr[left] <= arr[right]:  
            temp.push(arr[left])  
            left++
```



```

        else:
            temp.push(arr[right])
            count += mid - left + 1
            right++

    copy remaining left elements
    copy remaining right elements
    copy temp back into arr[low...high]

    return count

```

Important condition

Use:

```
arr[left] <= arr[right]
```

for the non-inversion case.

Equal elements are not inversions because an inversion requires:

```
arr[i] > arr[j]
```

not:

```
arr[i] >= arr[j]
```

Complexity

- Time: $O(n \log n)$
- Space: $O(n)$

Important

Use a 64-bit integer for the inversion count.

For a reverse-sorted array, the maximum inversions are:

```
n(n - 1) / 2
```

Revision key

During merge, if `left[i] > right[j]`, add all remaining elements in the left half:
`mid - i + 1`.

Arrays Part II — Final Revision Table

Problem	Optimal technique	Time	Extra space
Rotate Matrix	Transpose + reverse rows	$O(n^2)$	$O(1)$
Merge Intervals	Sort + merge sequentially	$O(n \log n)$	Output space
Merge Two Sorted Arrays	Gap method	$O((n+m) \log(n+m))$	$O(1)$
Find Duplicate	Floyd's cycle detection	$O(n)$	$O(1)$
Repeating and Missing	Math equations / XOR	$O(n)$	$O(1)$
Count Inversions	Modified merge sort	$O(n \log n)$	$O(n)$

One-Line Memory Tricks

Rotate Matrix:

Transpose first, then reverse every row.

Merge Intervals:

Sort and compare current start with previous merged end.

Merge Sorted Arrays:

Treat both arrays as one virtual array and reduce the gap.

Find Duplicate:

Duplicate number is the entrance of the linked-list cycle.

Repeating and Missing:

Use sum and square-sum equations, or divide XOR by a set bit.

Count Inversions:

During merge, add `mid-left+1` when the right value is smaller.

Pattern Recognition

Matrix transformation

→ Transpose and reverse

Overlapping ranges

→ Sort intervals first

Two sorted arrays with no extra space

→ Gap method

Values acting as valid indices

→ Floyd's cycle detection

One missing and one repeated value

→ Mathematical equations or XOR

Count pairs satisfying an order condition

→ Modified merge sort